

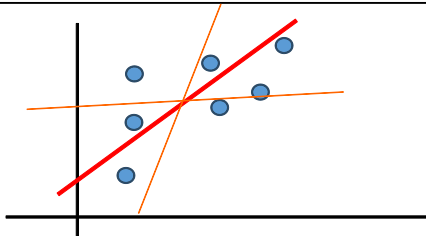
## Chapter 08 .사이킷런\_선형회귀/다항회귀

- LinearRegression() / 선형 회귀
- 다항회귀

## sklearn(사이킷런 : scikit-learn)

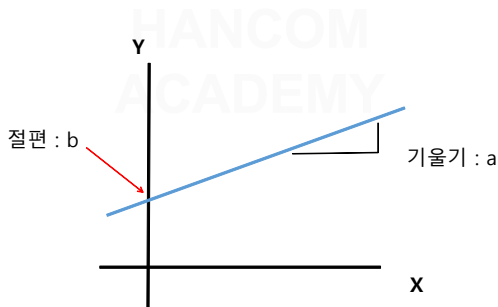
### • LinearRegression() : 선형 회귀

- 데이터를 선형 상관 관계로 모델링해 우리가 알고자 하는 값을 예측하는 방식
- 예를 들면 키와 몸무게 데이터를 선형 상관 관계로 모델링한 후 특정 사람의 키를 바탕으로 몸무게를 예측
- 관찰된 데이터를 기반으로 가장 거리가 짧은 데이터를 대표할 수 있는 선을 긋는 것이며, 그 선을 회귀식이라고 함
- 회귀식 가설함수  $H(x) = Wx + b$  (  $w$  : 가중치,  $b$  : 편향 )  
: 기울기  $W$ , 절편  $b$  에 따라 선의 모양이 정해지기 때문에 **관찰된 데이터와 오차가 가장 적은  $W, b$ 를 구함**으로써 새로운 데이터에 대한 예측을 수행할 수 있음



## sklearn(사이킷런 : scikit-learn)

- LinearRegression() : **선형 회귀**



$$Y = a * X + b$$

( 1차 방정식 )



머신러닝

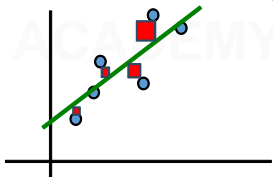
$$Y(\text{예측}) = Wx + b$$

훈련과정은 모델 파라미터인  
최적의 기울기(가중치)와 절편  
을 찾는 과정

## • LinearRegression() : 비용함수(손실함수)

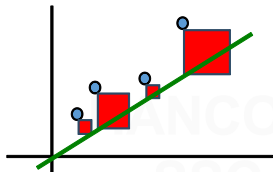
: 예측값과 실제 값의 오차가 가장 작아지는 해(w,b)를 찾는 목적으로 **평균제곱오차** 함수 사용

$$\text{square error(오차 제곱)} = (y_i - \underbrace{(Wx_i + b)}_{\text{예측값}})^2$$



1) 회귀 함수와 실제 데이터 간 거리

Vs



2) 회귀 함수와 실제 데이터 간 거리

$$\text{Mean Square Error(평균제곱오차)} = \frac{1}{n} \sum_{i=0}^n (y_i - (Wx_i + b))^2$$

회귀함수( $Wx_i + b$ ) 와 실제 데이터 사이의 거리(오차)를 제곱한 값을 평균낸 값

회귀의 목적 :  
위 1)회귀 함수처럼 **평균제곱오차**를 **최소화**하는것이라 정의



## • 비용함수(손실함수)의 최저점 찾는 방법

- MSE(평균제곱오차) 공식에서 유도된 **정규방정식(Normal Equation)**으로 직접 바로 계산
  - : 행렬 연산을 통해 한 번에 최적해를 구함
  - : 행렬 연산을 통해 결과를 바로 구하기 때문에 feature 의 크기가 커지면 계산 시간이 오래 걸림
  - : feature 의 크기가 적당한 수준일때 사용
  - ※ **sklearn의 Linear Regression 모델은 정규방정식(Normal Equation) 방식으로 최저점 계산**
  - ※ <https://gooopy.tistory.com/127> 참조
  - ※ <https://mazdah.tistory.com/831> 참조★
  - ※ <https://gyeo-ri.tistory.com/29> 참조
- MSE(평균제곱오차) 활용한 **경사하강법**
  - : 특정 지점에서 시작, 점점 값을 update해가면서 최저점을 찾음
  - : **feature** 의 크기가 지나치게 크다면 **선형 회귀에서도 경사하강법 사용 추천**
  - ※ **케라스 Sequential을 사용한 선형회귀 모델**

### 예제 1) 선형 회귀 활용 생선(농어) 무게 예측

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

perch_length=np.array([8.4,13.7,15.0,16.2,17.4,18.0,18.7,19.0,19.6,20.0,21.0,
21.0,21.0,21.3,22.0,22.0,22.0,22.0,22.0,22.5,22.5,22.7,
23.0,23.5,24.0,24.0,24.6,25.0,25.6,26.5,27.3,27.5,27.5,
27.5,28.0,28.7,30.0,32.8,34.5,35.0,36.5,36.0,37.0,37.0,
39.0,39.0,39.0,40.0,40.0,40.0,40.0,42.0,43.0,43.0,43.5,
44.0])

perch_weight=np.array([5.9,32.0,40.0,51.5,70.0,100.0,78.0,80.0,85.0,85.0,110.0,
115.0,125.0,130.0,120.0,120.0,130.0,135.0,110.0,130.0,
150.0,145.0,150.0,170.0,225.0,145.0,188.0,180.0,197.0,
218.0,300.0,260.0,265.0,250.0,250.0,300.0,320.0,514.0,
556.0,840.0,685.0,700.0,700.0,690.0,900.0,650.0,820.0,
850.0,900.0,1015.0,820.0,1100.0,1000.0,1100.0,1000.0,
1000.0])
```

```
train_input, test_input, train_target, test_target = \
    train_test_split(perch_length, perch_weight, random_state=42)

print(train_input.shape)
print(test_input.shape)
# 사이킷런 훈련을 위해 2차원 배열로 변환
train_input = train_input.reshape(-1,1) # 넘파이 배열 크기 자동 지정 : -1
test_input = test_input.reshape(-1,1)
print(train_input.shape)
print(test_input.shape)
lr_model = LinearRegression() #선형회귀 모델
```

```
#훈련
lr_model.fit(train_input, train_target)
```

```
#평가
print(lr_model.score(test_input, test_target))
```

```
#예측 : 50cm 농어 무게 예측 : 1241g 으로 예측
print(lr_model.predict([[50]]))
```

0.824750312331356

[1241.83860323]



# LinearRegression 클래스가 찾은 기울기, 절편 속성

#  $y = a * x + b$

#  $a$  : lr\_model.coef\_ ,  $b$  : lr\_model.intercept\_

print(lr\_model.coef\_, lr\_model.intercept\_)

[39.01714496] -709.0186449535474

# 훈련세트 산점도

plt.scatter(train\_input, train\_target)

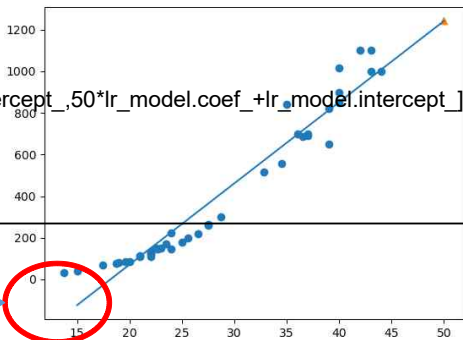
# 15 ~ 50 까지 직선 ,  $y = a * x + b$  적용

plt.plot([15,50],[15\*lr\_model.coef\_+lr\_model.intercept\_,50\*lr\_model.coef\_+lr\_model.intercept\_])

# 50 Cm 놓어 데이터 산점도

plt.scatter(50, 1241.8, marker='^')

plt.show()

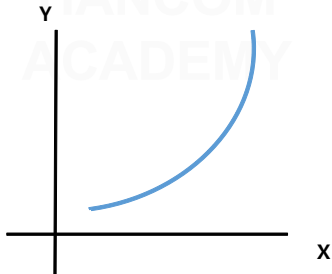


직선 선형회귀는 무게가 0g 이하로 내려가는 예측의 문제점 존재



- 직선 선형회귀 문제를 보완한 다항 회귀( 훈련 데이터 특성의 개수 증가 )
  - 다항식을 사용한 선형 회귀 : 다항 회귀

-  $y = a * x^2 + b * x + c$  : 2차 방정식



```
train_input, test_input, train_target, test_target = \
    train_test_split(perch_length, perch_weight, random_state=42)
```

```
print(train_input.shape)
```

```
print(test_input.shape)
```

# 사이킷런 훈련을 위해 2차원 배열로 변환

```
train_input = train_input.reshape(-1,1) # 넘파이 배열 크기 자동 지정 : -1
```

```
test_input = test_input.reshape(-1,1)
```

```
print(train_input.shape)
```

```
print(test_input.shape)
```

**# 길이에 제공한 특성 추가**

**# 특성 ( 길이제공, 길이 ) --> 타겟 (무게)**

```
train_poly = np.column_stack((train_input**2, train_input))
```

```
test_poly = np.column_stack((test_input**2, test_input))
```

```
print(train_poly.shape, test_poly.shape)
```

```
print(train_poly[:5])
```

(42, 2) (14, 2)

[[ 384.16 19.6 ]

[ 484. 22. ]

[ 349.69 18.7 ]

[ 302.76 17.4 ]

[1296. 36. ]]

```
lr_model = LinearRegression() # 모델 생성
```

```
# 훈련
```

```
lr_model.fit(train_poly, train_target)
```

```
# 예측
```

```
print( lr_model.predict([[50**2, 50]]) )
```

[1573.98423528]

```
# 훈련계수(가중치) , 절편 ( 2차원 방정식 )
```

```
print(lr_model.coef_, lr_model.intercept_)
```

```
#  $y = 1.01 x^2 + (-21.6) * x + 116.05$ 
```

```
# 훈련세트 산점도
```

```
plt.scatter(train_input, train_target)
```

```
# 2차방정식 그래프
```

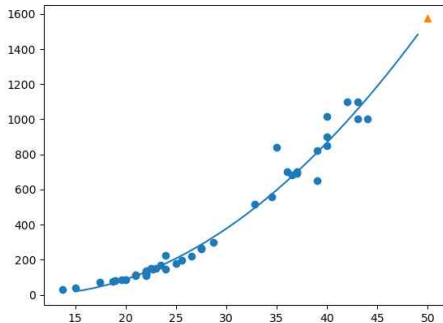
```
xpoint = np.arange(15,50)
```

```
plt.plot(xpoint, 1.01*xpoint**2 - 21.6*xpoint + 116.05)
```

```
# 50cm 놓어 데이터
```

```
plt.scatter(50, 1574, marker='^')
```

```
plt.show()
```



테스트 세트 평가 점수가 더 높은 과소적합 문제가 존재! ( 특성이 너무 단순 )

## • 특성 공학

- 특성 공학 : 기존 특성을 사용해 새로운 특성 생성, 다양한 특성 추가 하여 과소 적합 해결

## • 사이킷런 변환기 제공

- 특성을 만들거나 전처리 위한 다양한 클래스 제공
- 변환 클래스의 `fit()`, `transform()` 메서드 제공, `fit()+transform()` => `fit_transform()`
- **sklearn.preprocessing** 패키지 **PolynomialFeatures** 클래스 활용
- **fit()** : 모델 클래스와 달리 입력 데이터만 전달, 새롭게 만들 특성 조합을 찾음
- **transform()** : 실제 데이터를 변환

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly = PolynomialFeatures()  
poly.fit([[2,3]])  
print(poly.transform([[2,3]]))
```

2개의 특성

[[1. 2. 3. 4. 6. 9.]]

절편 특성

각 특성을 제곱한 항 추가, 특성끼리 곱한 항 추가

$2, 3, 2^2, 2*3, 3^2$

절편 특성 제거

`PolynomialFeatures(include_bias=False)`

사이킷런 모델은 자동으로 특성에 추가된 절편 항 무시함으로 `include_bias = False` 지정 안해도 됨!

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

```
fishdf = pd.read_csv('perch_fish_dataset.csv') # 길이 , 높이 , 두께
print(fishdf)
```

**# 특성 데이터 numpy data 변환**

```
perch_fulldata = fishdf.to_numpy()
print(perch_fulldata)
```

**# 타깃 데이터 ( 무게 )**

```
perch_weight = np.array([5.9, 32.0, 40.0, 51.5, 70.0, 100.0, 78.0, 80.0, 85.0, 85.0, 110.0,
    115.0, 125.0, 130.0, 120.0, 120.0, 130.0, 135.0, 110.0, 130.0,
    150.0, 145.0, 150.0, 170.0, 225.0, 145.0, 188.0, 180.0, 197.0,
    218.0, 300.0, 260.0, 265.0, 250.0, 250.0, 300.0, 320.0, 514.0,
    556.0, 840.0, 685.0, 700.0, 700.0, 690.0, 900.0, 650.0, 820.0,
    850.0, 900.0, 1015.0, 820.0, 1100.0, 1000.0, 1100.0, 1000.0,
    1000.0])
```

```
# perch_fulldata 과 perch_weight 데이터를 훈련세트 와 테스트 세트로 분리
train_input, test_input, train_target, test_target = \
    train_test_split(perch_fulldata, perch_weight, random_state=42)
```

```
print(train_input) # 3개의 특성 데이터를 갖음
print(train_input.shape)
```

# 특성 변환

```
poly = PolynomialFeatures(include_bias=False)
poly.fit(train_input)
train_poly = poly.transform(train_input)
print(train_poly[:5])
print(train_poly.shape) # 특성이 9개를 늘어남
```

(42, 9)

# 각 특성 조합 생성 방법을 출력

```
print( poly.get_feature_names() )
```

['x0', 'x1', 'x2', 'x0^2', 'x0\*x1', 'x0\*x2', 'x1^2', 'x1\*x2', 'x2^2']

# 테스트 세트도 변환 ( 훈련세트 기준으로 테스트 세트로 변환 )

```
test_poly = poly.transform(test_input)
```

```
print(test_poly[:5])
```

```
print(test_poly.shape)
```

(14, 9)

```
lr_model = LinearRegression()
```

# 변환 특성으로 훈련

```
lr_model.fit(train_poly, train_target)
```

# 변환 특성으로 평가

```
print(lr_model.score(train_poly, train_target))
```

```
print(lr_model.score(test_poly, test_target))
```

0.9903183436982124  
0.9714559911594199

늘어난 특성으로  
만족할 만한 모델 평  
가 결과를 얻음



But

#고차항특성추가, degree 매개변수=5: 5제곱까지 특성추가  
poly=PolynomialFeatures(degree=5, include\_bias=False)

위 처럼 degree 매개변수를 활용해서  
특성 개수를 늘렸지만 훈련데이터셋에 과대 적합  
되는 문제점 발생!!

# Thank You